
F1 Simulator – Individual Report

By

MINGYU YANG



Department of Computer Science
UNIVERSITY OF BRISTOL

A report submitted to the University of Bristol in accordance
with the requirements of the degree of MSC COMPUTER
SCIENCE in the Faculty of Engineering.

AUGUST 2026

Word count: ten thousand and four

Abstract

This report describes my individual contribution to an AI-driven racing simulation system built on TORCS and IBM Granite, focusing on the AI Race Engineer’s tyre wear estimation and proactive alert features. I designed a heuristic model that estimates tyre wear from speed and cornering data, since TORCS provides no such telemetry field, and a pit-strategy recommendation built on top of it. I also designed a proactive alert system that applies alarm-fatigue principles from human-computer interaction research to avoid repeatedly warning the driver about the same problem. A key technical challenge was that the underlying language model trusted pre-computed conclusions over raw telemetry data when both were placed in the same prompt; after testing an alternative fix and discussing it with my team, I resolved this by decoupling the estimation from the question-answering path entirely. The report also covers two features I built and later removed after they caused cross-feature data contamination, and the process by which the team and I decided to remove rather than repair them. Evaluation combines unit tests, a quantitative simulation of the wear model, and lessons from testing with a real language model rather than relying on code-level tests alone.

Author's declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

SIGNED: MINGYU YANG

DATE: [24/08/2026]

Table of Contents

	Page
List of Figures	iv
1 Introduction and Context	1
2 Innovation and Creativity	1
2.1 Innovation 1: A heuristic model for estimating tyre wear	1
2.2 Innovation 2: Designing proactive alerts around alarm fatigue	2
3 Technical Development	2
3.1 The feature I built	2
3.2 Implementation details	3
3.3 Challenge one: should tyre estimation be integrated into the team’s shared architecture?	3
3.4 Challenge two: the model trusted a wrong conclusion over raw data	4
3.5 Where this can still be improved	5
4 Evaluation Skills	6
4.1 Unit tests: a matrix of boundary conditions	6
4.2 Quantitative evaluation: running the wear model over real data	6
4.3 What real-model testing exposed	7
5 Project Management Skills	7
5.1 The boundary I set for myself, and how the team coordinated	7
5.2 My own weekly rhythm	7
6 Critical Self-Evaluation and Future Work	8
6.1 Two features I built and later removed	8
6.2 The problem I found	8
6.3 The discussion and the final decision	8
6.4 What happened after removal	9
6.5 An area I think can still be improved further	9
6.6 Reflection and future work	9
Bibliography	10

List of Figures

Figure	Page
2.1 Proactive alert: edge-triggered de-duplication. A problem only triggers a broadcast on the transition into it; while the same problem persists across ticks, or once it clears back to “none”, nothing repeats.	3
3.1 <code>RaceStrategyTracker.update()</code> state machine: the three logic lines inside one <code>update()</code> call – pit-stop reset (fuel jump), wear accumulation (speed + cornering), and new-lap detection (<code>lap_time</code> reset) for per-lap statistics.	4

1 Introduction and Context

This project is a team effort to build an AI-driven racing simulation system on top of TORCS, powered by IBM Granite (full project background and the four-strand team split are covered in the Group Report). I was responsible for one strand, the “AI Race Engineer” – a conversational feature that answers a driver’s questions in real time (for example, “should I pit now?”). The most foundational parts of this strand – the chat endpoint, the base text of the AI persona, and the floating debug client – were originally built by my teammate kenny; I built on top of this already-working foundation.

The motivation is straightforward: during a race, a driver has no time to study a dashboard full of numbers, let alone work out when to pit. This is not just intuition – Wickens’ Multiple Resource Theory [4] explains why: controlling the car and reading/interpreting numeric gauges both draw on the same visual and cognitive resources, so the two tasks compete and performance suffers. I noticed this myself during testing – typing out a question while still trying to drive was a clear source of distraction, which is exactly why this feature needed to work as “ask directly, get a direct answer” rather than making the driver dig through data themselves.

2 Innovation and Creativity

This section covers two design decisions that I think genuinely demonstrate creativity – both were solutions I worked out myself with no existing precedent to follow within the project.

2.1 Innovation 1: A heuristic model for estimating tyre wear

TORCS’s telemetry does not include a tyre wear field at all – speed, RPM and fuel are all there, but wear is not. Yet one of the questions drivers ask most often is exactly “how are my tyres doing?” The AI persona explicitly requires that it never invents numbers, so without real data, this question was originally impossible to answer.

My solution was to design a wear estimation model that infers wear from the data that is already available: the faster the car is going, and the more cornering it does (approximated using the change in lateral track position), the faster tyre wear should accumulate. This approach draws on the basic logic of Archard’s wear law from tribology [1] – that wear volume is proportional to contact pressure and sliding distance – transplanting a classical principle from physics into a completely different context, game telemetry, to fill a data gap the game itself does not provide. This is obviously not a precise physical simulation – TORCS itself does not model real tyre ageing – so what I built is a self-consistent estimate the AI can reason from,

not an accurate model.

2.2 Innovation 2: Designing proactive alerts around alarm fatigue

Beyond answering questions reactively, I also built a proactive alert feature: off by default, but when enabled, the AI will speak up on its own if the car goes off track, or if tyre wear or fuel reach a critical level, without the driver having to ask. The creativity here is not in the idea of “proactive alerts” itself, but in borrowing a design principle from an entirely different discipline to solve a specific engineering problem.

While designing this feature I drew on the design principle of “alarm fatigue” – a concept well established in the human-computer interaction literature for medical devices and aircraft cockpits. Cvach’s systematic review of clinical alarm systems [2] notes that overly frequent alarms cause users to gradually become desensitised to all alarms, so that genuinely dangerous alerts end up being ignored too. I applied this design idea, originally used for medical alarm systems, creatively to a completely different domain, racing simulation: alerts are split into “physical” (e.g. going off track, which needs immediate attention) and “strategic” (e.g. it’s time to pit, which is less urgent) priority tiers, and the same problem is only reported once, rather than repeatedly firing while the underlying state briefly fluctuates.

I ran into exactly this problem during testing – the same “get back on track” alert fired five times within thirty seconds, because the car was drifting on and off the track edge repeatedly and each excursion was treated as a new hazard. This real-world test directly confirmed the need for the alarm-fatigue design principle and drove the addition of the edge-triggered de-duplication logic described above; the decision flow is shown in Figure 2.1.

3 Technical Development

This section focuses on a single feature – tyre wear estimation and pit strategy – going into more depth on my specific contribution, the technical challenges I ran into, and how each was ultimately resolved.

3.1 The feature I built

I independently designed and implemented the complete tyre wear estimation and pit strategy recommendation feature. Its core is a stateful tracker called `RaceStrategyTracker` (state machine shown in Figure 3.1), together with the pit-window recommendation logic `estimate_pit_window`, and close to 20 accompanying unit tests (covered in Chapter 4).

Proactive alert: edge-triggered de-duplication

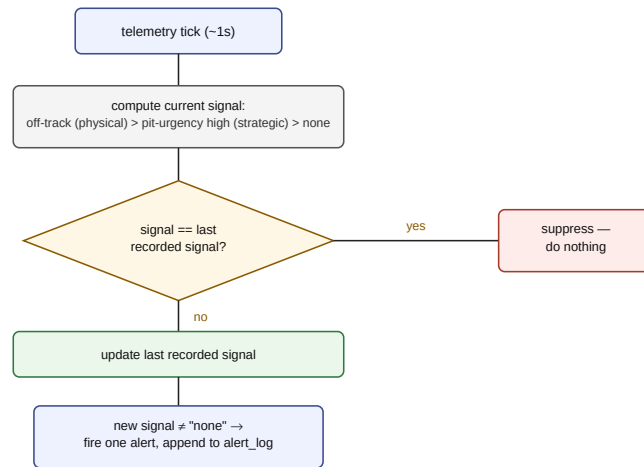


Figure 2.1: Proactive alert: edge-triggered de-duplication. A problem only triggers a broadcast on the transition into it; while the same problem persists across ticks, or once it clears back to “none”, nothing repeats.

3.2 Implementation details

`RaceStrategyTracker` has no direct way of knowing “when the car just pitted”, because TORCS provides no such signal, so I used an indirect heuristic instead: under normal driving fuel only ever decreases, so if a reading shows fuel jumping up sharply compared to the previous reading, that is a reliable signal that a pit stop with a refuel just happened, and the tyres have been changed too – at which point I reset the wear counter to zero. Similarly, the game gives no lap counter, so I use a sharp drop in lap time back toward zero as the signal that a new lap has started, and use that boundary to work out roughly how much fuel and tyre wear was consumed over the previous lap. Once wear percentage and per-lap consumption are known, `estimate_pit_window` checks two things at once – whether tyre wear has crossed a warning threshold, and how many laps of fuel remain at the current burn rate – and recommends a pit stop with “medium” or “high” urgency if either one enters the danger zone.

3.3 Challenge one: should tyre estimation be integrated into the team’s shared architecture?

Once the core logic was working, the first technical challenge I faced was architectural: should tyre estimation be integrated into the `car_state` data structure and the conversation context manager class that the whole team shares, making it part of the project’s common architecture, or should it be kept fully independent, reading shared data but never writing to it?

I brought this question to our weekly team meeting for discussion. Tyre wear is a problem

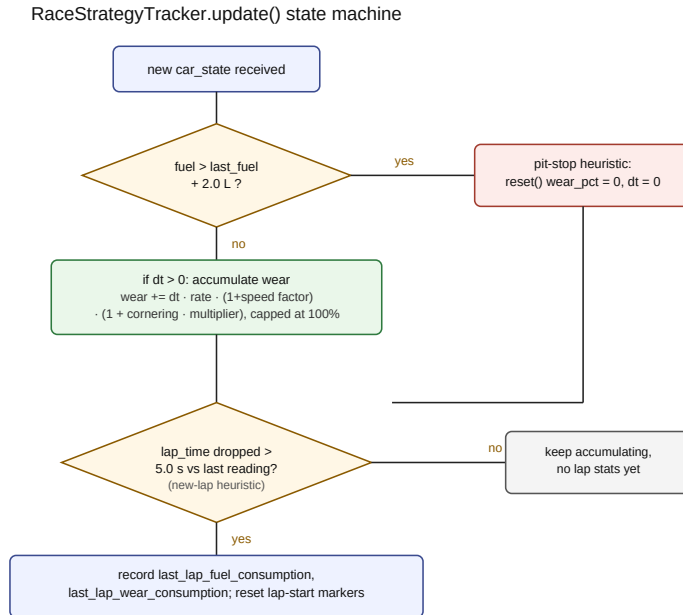


Figure 3.1: `RaceStrategyTracker.update()` state machine: the three logic lines inside one `update()` call – pit-stop reset (fuel jump), wear accumulation (speed + cornering), and new-lap detection (`lap_time` reset) for per-lap statistics.

unique to my strand – no other direction needed this data – so “making it easier for others to reuse” was never a valid justification. The real concern was whether integrating into shared code would risk interfering with teammates’ work in progress. After discussion, the team agreed that, given nobody was dedicated to maintaining a shared architecture at the time and everyone was developing in parallel, the risk of conflicts from touching shared code outweighed the benefit of architectural consistency. The final decision was to keep `tire_strategy.py` fully independent: it only reads the `car_state` format we had already agreed on as a team, and never writes to or modifies any shared code path, so that even if this module has a bug, it cannot affect a teammate’s feature. The trade-off is that this feature has remained an isolated part of the codebase that I alone maintain, never becoming a genuinely shared architectural component – a point I return to in Chapter 6.

3.4 Challenge two: the model trusted a wrong conclusion over raw data

The second challenge appeared after the feature went live: real-model testing exposed a problem. I had originally been placing the computed wear percentage and pit recommendation directly into the prompt sent to the model, alongside the raw telemetry data, and letting the model reason over all of it together. But the model turned out to trust the “conclusion” I had pre-

computed in the prompt more than the raw telemetry figures in that same prompt, even when the two contradicted each other – producing inaccurate or self-contradictory answers as a result.

Once I found this, my first attempt at a fix was to strengthen the prompt instruction itself, adding a line such as “treat the following raw telemetry line as authoritative, and ignore any inference that appears to contradict it.” I also brought this option to the weekly team meeting, and testing showed it was not a good fix – simply changing the wording of an instruction in the prompt could shift the model’s behaviour across the board, not just on this one issue; the tone and accuracy of answers to completely unrelated questions could shift too. After discussion, the team agreed that this unpredictability was more concerning than the original specific bug, and that continuing to patch the prompt was not a sustainable approach. The final decision was to remove this estimation entirely from the question-answering prompt and expose it instead through an independent endpoint (`GET /api/engineer/tire_estimate`), polled by the front end every few seconds and displayed separately – a completely different path from the chat flow, with no interference between the two.

Behind this decision is a more general design pattern: rather than letting the model compare numbers and reach judgements itself, compute the conclusion in plain Python code first, and let the model only phrase that conclusion into a natural sentence. I did not invent this pattern – it had already been validated by the teammate responsible for telemetry analysis in `race_analyzer.py` – and I reused the same approach here for tyre estimation, and again for the data-gap fallback logic covered in Chapter 4.

3.5 Where this can still be improved

After the rework described above, the tyre wear and pit strategy feature was not cut – it still works today through the independent `GET /api/engineer/tire_estimate` endpoint, and the dashboard has a live-updating card driven by the same tracker, so the driver can always see the current wear estimate.

What is not yet connected is the path of asking about tyres directly in conversation: if a driver now asks “how are my tyres doing?” in the chat box, the answer follows the data-gap fallback logic covered in Chapter 4 and returns “I don’t have that data,” rather than reading the value behind that dashboard card. This does not fully line up with the central scenario introduced at the start of Chapter 1 – a driver directly asking “should I pit now?” and the AI answering with estimated data – since that scenario currently has to be satisfied by looking at the dashboard card rather than by asking the AI. Ideally, the chat path should be able to read the tracker’s current value once when it recognises a tyre-related question, and return that directly as the answer, without feeding it into the prompt the model compares against raw data – avoiding the risk described above, while giving the driver the same figure whichever way they

access it.

4 Evaluation Skills

4.1 Unit tests: a matrix of boundary conditions

I wrote close to 20 unit tests for `tire_strategy.py`, with around 13 of them specifically for `estimate_pit_window`, deliberately covering a matrix of boundary conditions rather than just a handful of normal cases. Two concrete examples: `test_critical_fuel_overrides_medium_tire_urgency_to_high` checks that when fuel has reached a critical level, the final recommendation is escalated to “high” even if tyre urgency is only “medium,” because running out of fuel is more urgent than tyre wear; the reverse case, `test_medium_fuel_does_not_downgrade_an_already_high_tire_urgency`, checks that when fuel is only moderately tight, it must not downgrade a tyre urgency that is already “high.” Together, these two tests confirm that the rule “take whichever is more urgent” holds in both directions.

4.2 Quantitative evaluation: running the wear model over real data

Saying “the unit tests pass” does not, on its own, tell you whether the numbers the model produces are actually sensible, so I wrote a separate script to run `RaceStrategyTracker` over a complete simulated driving sequence. It should be noted that this is a synthetic telemetry sequence I constructed myself (constant speed of 180 km/h, mild cornering oscillation, roughly 2.2 L of fuel burned per lap, 80-second laps), not data recorded from a real TORCS session, since we had no existing real-race telemetry log I could run this analysis against.

Over 30 simulated laps: wear escalated from “low” to “medium” urgency (65% wear) at lap 11, and from “medium” to “high” (85% wear) at lap 14 – roughly a three-lap buffer between the two. After the simulated pit stop reset at lap 15, the same pattern repeated, with the second “medium”-to-“high” gap also lasting three laps, showing the model gives a consistent warning window for the same input pattern. These figures show that the model’s warning window is stable and predictable, giving the driver roughly three laps of reaction time to plan a pit stop before wear actually reaches the danger zone – which I take as evidence that this feature meets the quality bar I was aiming for. That said, this evaluation is not exhaustive – I did not test more extreme driving styles such as hard acceleration or braking, and it has not been validated against real player data.

4.3 What real-model testing exposed

The problem described in Section 3.4 – the model trusting a computed conclusion over raw data – was only discoverable through real-model testing, not unit testing: unit tests check whether the code logic is correct, but they cannot test how the model actually “understands” or “uses” the information it is given. This is also where LLM-driven features differ sharply from traditional game-logic testing: conventional testing assumes the same input always produces the same output, but a model’s behaviour is inherently non-deterministic. There is dedicated research on how to test the behaviour of LLM-driven systems, such as the CheckList method proposed by Ribeiro et al. [3], which argues that accuracy alone is not enough and that model behaviour needs to be tested systematically across specific scenarios. I applied the same principle in `test_feature_apis.py`, adding integration tests that check the actual prompt content sent to the model – for example, confirming it no longer contains the words “tyre wear” after the decoupling in Section 3.4, and that a tyre-wear question now goes through the same fallback flow as any other unavailable-data topic.

5 Project Management Skills

5.1 The boundary I set for myself, and how the team coordinated

Our team agreed that each of us would develop our own assigned feature and avoid touching shared logic others were actively maintaining (the team’s overall workflow, including the weekly meeting rhythm, is described in the Group Report). The specific rule I set for myself was to only change the parts directly related to the “engineer Q&A” feature. Decisions such as how I sequenced tyre estimation against proactive alerts, both of the architectural decisions covered in Chapter 3, and later whether to remove the event-memory and priority modules once problems were found, were all things I brought to and synced with the team at our weekly meetings – not calls I made alone.

5.2 My own weekly rhythm

Within that team structure, I planned my own work week by week rather than jumping between tasks. The specific weekly plan in July was: week one, tyre wear estimation; week two, proactive alerts; week three, event memory; week four, priority judgement built on top of event memory. These features were developed and validated locally in July, but not pushed to the repository until early August – I was still testing locally on a small scale and had not reached a version I considered final, so I was in no hurry to push code that was still being changed.

Not long after finishing priority judgement, I discovered that the event-memory module itself had a problem (covered in Chapter 6). This weekly rhythm meant that before starting each new feature I could check whether the previous week’s work had exposed any problems – it was exactly this rhythm that let the priority-judgement issue be caught as soon as it appeared, rather than only being dealt with after everything had piled up.

6 Critical Self-Evaluation and Future Work

6.1 Two features I built and later removed

Beyond the features described above that are still in use, I also built two features that I later removed myself: “event memory,” which kept a record of recent incidents and conversation from the race, and “priority judgement,” which decided which of several simultaneous issues should be mentioned first, drawing on data accumulated by event memory.

6.2 The problem I found

There was a dependency between these two modules: priority judgement, when making its combined decision, would draw on the contextual data accumulated by event memory. That is exactly where the problem originated – event memory kept accumulating data continuously, and once the context contained incorrect or irrelevant Q&A records, that bad data directly affected the accuracy of subsequent model answers; because priority judgement depended on this same data, the effect of the corruption was amplified and propagated into more features. This also made testing very difficult: after these two modules were built, my teammates’ tests essentially all failed, and the cause was this kind of cross-scenario data contamination – not a single mis-written test case, but a test environment whose state was already dirty from the start.

6.3 The discussion and the final decision

Once this problem surfaced, I raised it at our weekly meeting, and we went back and forth on it for a while as a team: one option was to fix the underlying state-management bug in both modules and keep the features; the other was to remove them entirely and replace them with a simpler, stateless approach. I was initially in favour of fixing the bug and keeping the functionality, since I thought both features were genuinely interesting. But on further reflection, given how complex both modules already were, even fixing this particular bug would leave the

same kind of state contamination likely to resurface with any future feature – a fix that would only treat the symptom, not the cause. After weighing this up, we decided to remove both modules and reimplement the underlying need in a simpler way.

6.4 What happened after removal

Once priority judgement was removed, the proactive alert feature that had depended on it could no longer work as before, so I rewrote it as a stateless, direct judgement: each check only looks at the current frame of data, and fires if it matches either a “physical danger” or “strategic urgency” condition, with no need to remember anything that happened previously. This means that even if something goes wrong again in future, the blast radius is limited to that single judgement, rather than carrying corrupted state forward into the rest of the race as before.

6.5 An area I think can still be improved further

The event-memory and priority-judgement issues above have already been dealt with. There is still one area on the tyre-strategy side that I think could be taken further: at present, a driver can only see the tyre wear estimate through the dashboard card, and cannot ask for it directly in conversation (see Section 3.5 for the detail). At that same point in the project I had the data-gap fallback logic and UI improvements to work on, and never found the time to go back and design a path for “asking about tyres in chat” that would not reintroduce the risk described in Section 3.4. Given more time, I would want the chat path to read the tracker’s current value once when it recognises a tyre-related question and return it directly as the answer, rather than feeding it into the prompt the model compares against raw data – though I have not yet had time to implement or verify this.

6.6 Reflection and future work

Looking back, event memory and priority judgement were probably heading in the wrong direction from the start of their design – a simpler, stateless approach would have covered most of the actual need, and would have been far less prone to bugs. If I were doing this again, I would build the simplest version first, and only add stateful memory once I had concrete evidence that the simple version was not enough. The tyre wear model could similarly be made more precise by incorporating more signals, such as the curvature of different corners rather than just the change in lateral position; the proactive alert system could be developed into finer-grained severity levels than the current “alert”/“don’t alert” split; and if tyre estimation is ever meant to become part of the project’s shared architecture, the data-interface conventions would need to be agreed with the rest of the team first, rather than everyone continuing to build their own separate pieces as at present.

Bibliography

- [1] J. F. ARCHARD, *Contact and rubbing of flat surfaces*, Journal of Applied Physics, 24 (1953), pp. 981–988.
- [2] M. CVACH, *Alarm fatigue: A systematic review*, Biomedical Instrumentation & Technology, 46 (2012), pp. 268–277.
- [3] M. T. RIBEIRO, T. WU, C. GUESTRIN, AND S. SINGH, *Beyond accuracy: Behavioral testing of NLP models with CheckList*, in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 4902–4912.
- [4] C. D. WICKENS, *Multiple resource theory: A new approach to human information processing*, Theoretical Issues in Ergonomics Science, 3 (2002), pp. 159–177.